

Mosaic Problem Analysis: Implementation Pitfalls

The Mosaic problem (IOI 2024) is a complex grid problem involving prefix sums and diagonal mapping. The following table summarizes the key areas where the implementation failed to meet requirements, leading to partial scores.

Issue Category	Description	Impact
Memory Initialization	Pre-allocating arrays with fixed sizes (e.g., using '4' as a loop boundary) instead of dynamic N-based bounds.	Causes Segmentation Faults for small test cases ($N < 4$).
Loop Boundaries	Loops iterating to $< N$ instead of $\leq N$, missing the last row/column in prefix sum calculations.	Results in Wrong Answers for subrectangles that touch the bottom or right edges of the grid.
Access Patterns	Accessing $G[i][j]$ where i or j exceeds the allocated jagged array bounds.	Causes Undefined Behavior and program crashes.

Key Algorithmic Takeaways

- Jagged Array Handling:** When N is small, code must adapt dynamically rather than assuming a minimum N of 3 or 4.
- Prefix Sum Correctness:** Always verify that prefix sum arrays cover the full range of potential queries $[0, N-1]$ by sizing them to $N+1$.
- Mapping Logic:** The use of $j - i + \text{offset}$ is an efficient way to map diagonal elements to a 1D array without the performance overhead of `std::map`.

Recommended Coding Practices for Future Tasks

- Boundary Testing:** Always test implementation with the smallest possible constraints ($N=1$, $N=2$) before submitting.
- Memory Safety:** Use `std::vector` bounds-checking or explicit `if (i < N)` checks inside loops to prevent out-of-bounds access.
- Modular Verification:** Verify the logic for corner cases (e.g., subrectangles at $(0,0)$ or $(N-1, N-1)$) separately from the main logic.